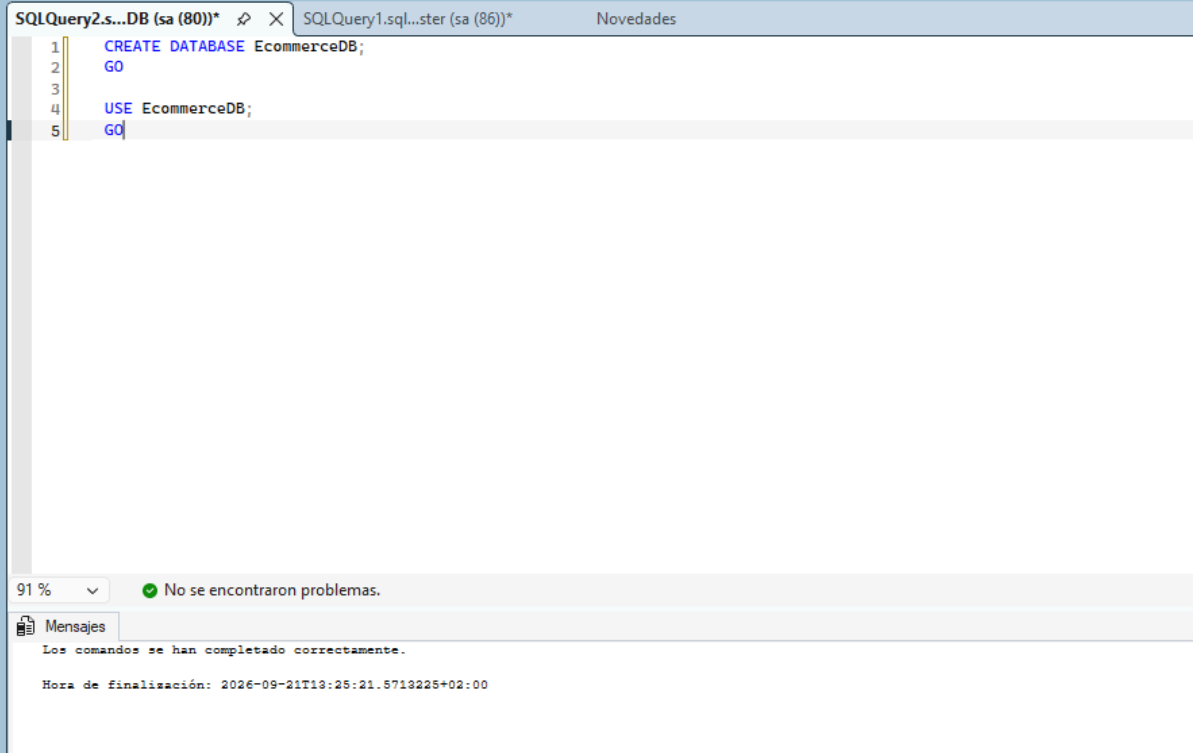


Ejercicio 1: Creación de la base de datos

Se ha creado la base de datos principal **EcommerceDB** que servirá como contenedor para todos los objetos del esquema de la plataforma de comercio electrónico.

Código SQL ejecutado:



The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays the following SQL commands:

```
1 CREATE DATABASE EcommerceDB;  
2 GO  
3  
4 USE EcommerceDB;  
5 GO
```

The bottom pane shows the execution progress at 91% and a message: "No se encontraron problemas." (No problems were found). Below this, a "Mensajes" (Messages) pane displays the following text:

Los comandos se han completado correctamente.
Hora de finalización: 2026-09-21T13:25:21.5713225+02:00

Ejercicio 2: Creación de tablas principales y restricciones (Constraints)

Se implementaron las tablas fundamentales (**Supplier**, **Category**, **Product**) con sus respectivas claves primarias e índices. Además, se añadieron restricciones de validación (**CHECK**) para asegurar que los precios y el stock no sean negativos, así como claves foráneas para mantener la integridad referencial. Finalmente, se insertaron datos de prueba.

Código SQL ejecutado:

SQLQuery2.s...DB (sa (80))* SQLQuery1.sql...ster (sa (86))* Novedades

```
1  USE EcommerceDB;
2  GO
3
4  -- Insert sample suppliers
5  INSERT INTO Supplier (SupplierName, Country, Email, Phone)
6  VALUES
7      ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
8      ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');
9
10 -- Insert sample categories
11 INSERT INTO Category (CategoryName, Description)
12 VALUES
13     ('Electronics', 'Electronic devices and accessories'),
14     ('Clothing', 'Apparel and fashion items');
15
16 -- Insert sample products
17 INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
18 VALUES
19     ('Wireless Mouse', 1, 1, 29.99, 100),
20     ('Cotton T-Shirt', 2, 2, 19.99, 250);
21 GO
```

91 % 14 0

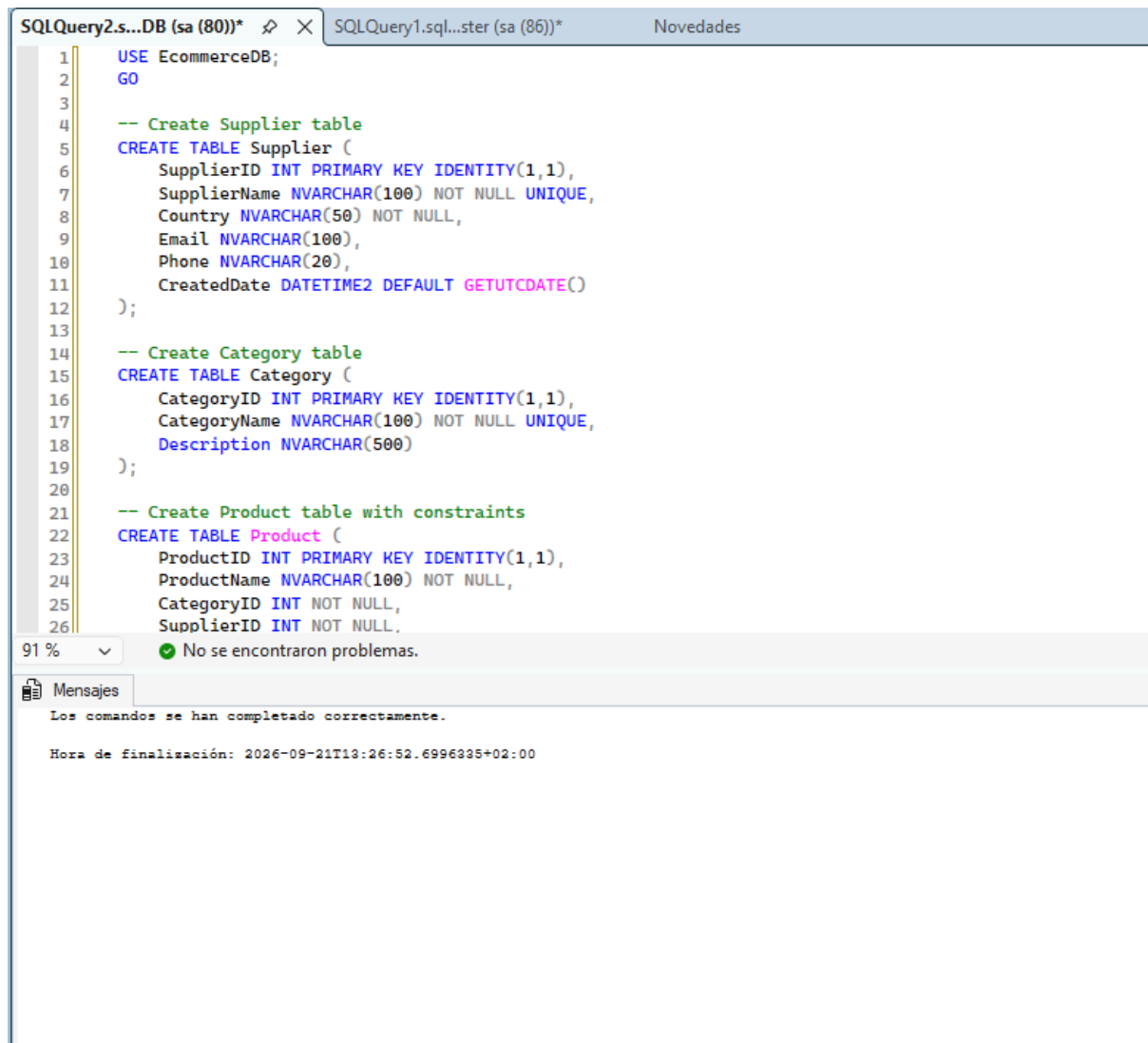
Mensajes

(2 filas afectadas)

(2 filas afectadas)

(2 filas afectadas)

Hora de finalización: 2026-09-21T13:27:44.1724548+02:00



```
1  USE EcommerceDB;
2  GO
3
4  -- Create Supplier table
5  CREATE TABLE Supplier (
6      SupplierID INT PRIMARY KEY IDENTITY(1,1),
7      SupplierName NVARCHAR(100) NOT NULL UNIQUE,
8      Country NVARCHAR(50) NOT NULL,
9      Email NVARCHAR(100),
10     Phone NVARCHAR(20),
11     CreatedDate DATETIME2 DEFAULT GETUTCDATE()
12 );
13
14 -- Create Category table
15 CREATE TABLE Category (
16     CategoryID INT PRIMARY KEY IDENTITY(1,1),
17     CategoryName NVARCHAR(100) NOT NULL UNIQUE,
18     Description NVARCHAR(500)
19 );
20
21 -- Create Product table with constraints
22 CREATE TABLE Product (
23     ProductID INT PRIMARY KEY IDENTITY(1,1),
24     ProductName NVARCHAR(100) NOT NULL,
25     CategoryID INT NOT NULL,
26     SupplierID INT NOT NULL,
```

91 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-21T13:26:52.6996335+02:00

Ejercicio 3: Implementación de tabla temporal para historial de precios

Se creó una tabla temporal (**ProductPrice**) con la opción **SYSTEM_VERSIONING = ON**. Esto permite a SQL Server mantener automáticamente un registro de auditoría completo de los cambios de precio, guardando el rango de tiempo en el que cada precio fue válido sin necesidad de programar triggers.

Código SQL ejecutado:

SQLQuery2.s...DB (sa (80))*

SQLQuery1.sql...ster (sa (86))*

Novedades

1

USE EcommerceDB;

2

GO

3

4

-- Query price history

5

SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime

6

FROM ProductPrice

7

FOR SYSTEM_TIME ALL

8

WHERE ProductID = 1;

91 %

6

0

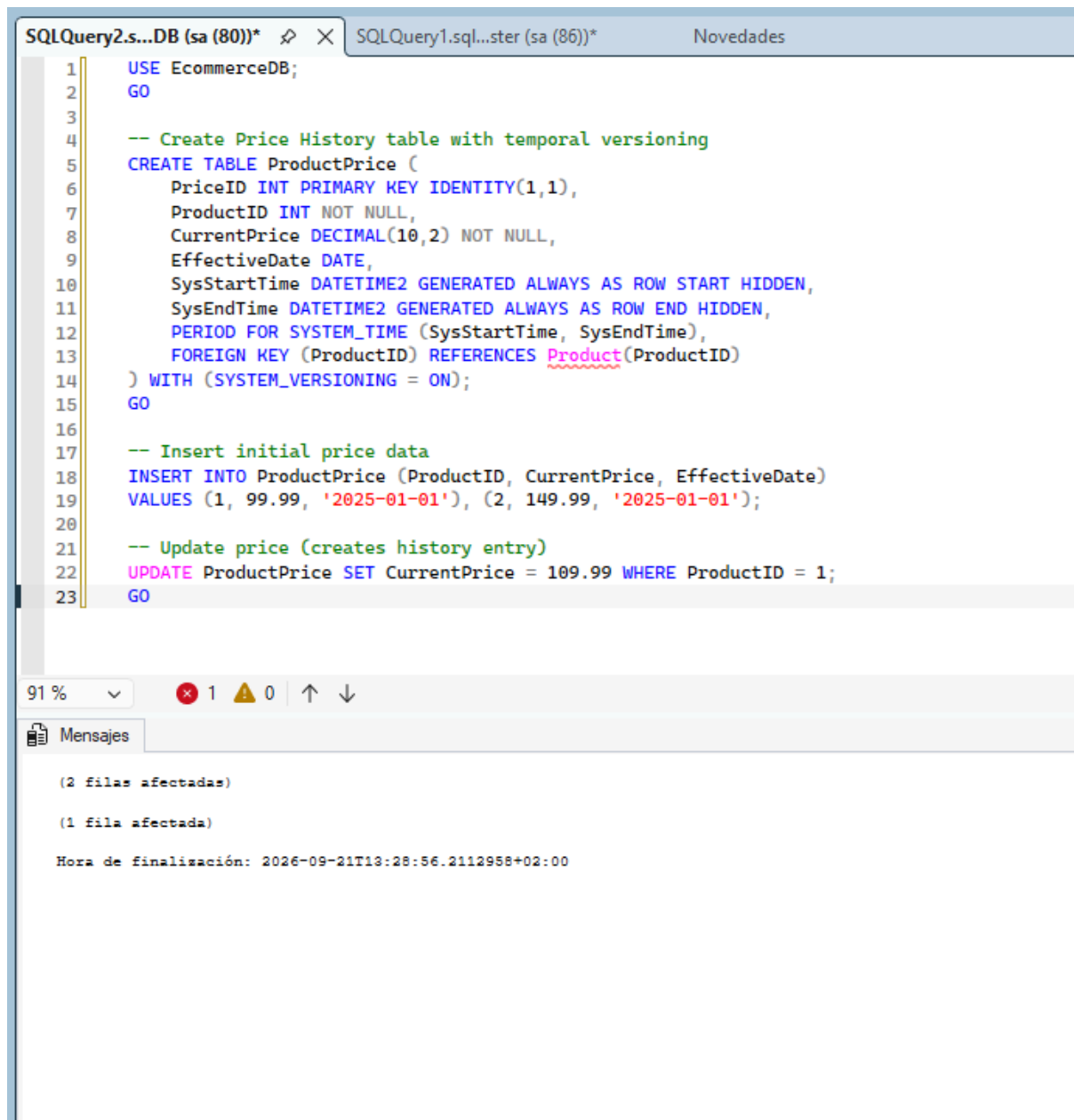
↑

↓

Resultados

Mensajes

	ProductID	CurrentPrice	SysStartTime	SysEndTime
1	1	109.99	2026-09-21 11:28:56.2062642	9999-12-31 23:59:59.9999999



The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays a SQL query window titled 'SQLQuery2.s...DB (sa (80))*'. The query is as follows:

```
1 USE EcommerceDB;
2 GO
3
4 -- Create Price History table with temporal versioning
5 CREATE TABLE ProductPrice (
6     PriceID INT PRIMARY KEY IDENTITY(1,1),
7     ProductID INT NOT NULL,
8     CurrentPrice DECIMAL(10,2) NOT NULL,
9     EffectiveDate DATE,
10    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,
11    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,
12    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
13    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
14 ) WITH (SYSTEM_VERSIONING = ON);
15 GO
16
17 -- Insert initial price data
18 INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
19 VALUES (1, 99.99, '2025-01-01'), (2, 149.99, '2025-01-01');
20
21 -- Update price (creates history entry)
22 UPDATE ProductPrice SET CurrentPrice = 109.99 WHERE ProductID = 1;
23 GO
```

The bottom pane shows a messages window titled 'Mensajes'. It displays the following output:

```
(2 filas afectadas)
(1 fila afectada)
Hora de finalización: 2026-09-21T13:28:56.2112958+02:00
```

Ejercicio 4: Almacenamiento flexible con columnas JSON

Se modificó la tabla `Product` para soportar metadatos variables utilizando el tipo de datos nativo `JSON`. Se agregó una columna calculada extrayendo el atributo `color` y se indexó para optimizar las búsquedas sobre datos no estructurados.

Código SQL ejecutado:

SQLQuery2.s...DB (sa (80))*

SQLQuery1.sql...ster (sa (86))*

Novedades

```
1  USE EcommerceDB;
2  GO
3
4  -- Query JSON data
5  SELECT
6      ProductID,
7      ProductName,
8      JSON_VALUE(Metadata, '$.color') AS Color,
9      JSON_VALUE(Metadata, '$.size') AS Size,
10     JSON_VALUE(Metadata, '$.material') AS Material
11  FROM Product
12  WHERE JSON_VALUE(Metadata, '$.color') = 'blue';
```

91 %

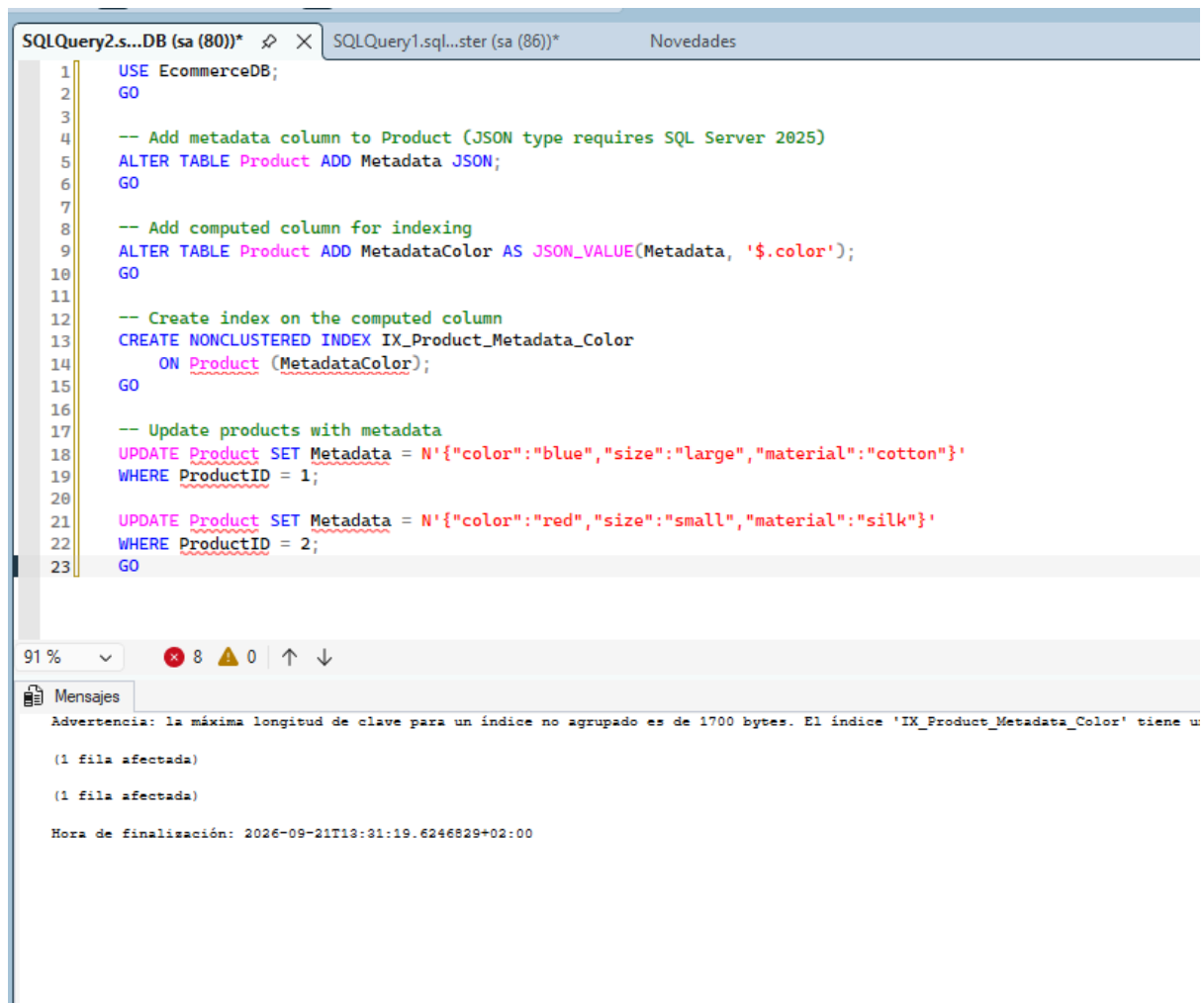
7 0

↑ ↓

Resultados

Mensajes

	ProductID	ProductName	Color	Size	Material
1	1	Wireless Mouse	blue	large	cotton



```
1 USE EcommerceDB;
2 GO
3
4 -- Add metadata column to Product (JSON type requires SQL Server 2025)
5 ALTER TABLE Product ADD Metadata JSON;
6 GO
7
8 -- Add computed column for indexing
9 ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
10 GO
11
12 -- Create index on the computed column
13 CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
14 ON Product (MetadataColor);
15 GO
16
17 -- Update products with metadata
18 UPDATE Product SET Metadata = N'{"color":"blue","size":"large","material":"cotton"}'
19 WHERE ProductID = 1;
20
21 UPDATE Product SET Metadata = N'{"color":"red","size":"small","material":"silk"}'
22 WHERE ProductID = 2;
23 GO
```

91 % 8 0

Mensajes

Advertencia: la máxima longitud de clave para un índice no agrupado es de 1700 bytes. El índice 'IX_Product_Metadata_Color' tiene u

(1 fila afectada)

(1 fila afectada)

Hora de finalización: 2026-09-21T13:31:19.6246829+02:00

Ejercicio 5: Particionamiento de la tabla de pedidos

Para mejorar el rendimiento y mantenimiento sobre grandes volúmenes de datos, se configuró una función de partición por trimestres y un esquema de partición. Finalmente, se aplicó a la nueva tabla **Order**, alineando su clave primaria y su índice con los rangos de fechas.

Código SQL ejecutado:

SQLQuery2.s...DB (sa (80))*

SQLQuery1.sql...ster (sa (86))*

Novedades

1

USE EcommerceDB;

2

GO

3

4

-- Query by partition

5

SELECT

6

\$PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,

7

COUNT(*) AS OrdersInPartition,

8

MIN(OrderDate) AS MinDate,

9

MAX(OrderDate) AS MaxDate

10

FROM [Order]

11

GROUP BY \$PARTITION.PF_OrderDate(OrderDate);

91 %

1

0

↑

↓

Resultados

Mensajes

	PartitionNumber	OrdersInPartition	MinDate	MaxDate
1	2	2	2025-01-15	2025-02-20
2	3	1	2025-06-10	2025-06-10

```
SQLQuery2.s...DB (sa (80))* x SQLQuery1.sql...ster (sa (86))* Novedades
13
14 -- Create partitioned Order table
15 -- Include OrderDate in primary key for clustered index alignment
16 CREATE TABLE [Order] (
17     OrderID BIGINT IDENTITY(1,1),
18     OrderDate DATE NOT NULL,
19     CustomerName NVARCHAR(100) NOT NULL,
20     TotalAmount DECIMAL(12,2) NOT NULL,
21     OrderStatus NVARCHAR(20) DEFAULT 'Pending',
22     CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
23     CHECK (TotalAmount > 0),
24     CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'))
25 ) ON PS_OrderDate(OrderDate);
26
27 -- Create partitioned index
28 CREATE NONCLUSTERED INDEX IX_Order_Customer
29 ON [Order](CustomerName)
30 ON PS_OrderDate(OrderDate);
31 GO
32
33 -- Insert sample orders
34 INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES
35 ('2025-01-15', 'John Smith', 299.97, 'Delivered'),
36 ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
37 ('2025-06-10', 'Bob Johnson', 449.95, 'Processing');
38 GO
91 % No se encontraron problemas. Línea: 38, Cará
Mensajes
Se ha creado correctamente el esquema de partición 'PS_OrderDate'. 'PRIMARY' tiene la marca de grupo de archivos usado a continuación en el esquema de partición 'PS_OrderDate'.
(3 filas afectadas)
Hora de finalización: 2026-09-21T13:33:09.8352779+02:00
91 % No se encontraron problemas. Línea: 6, Carácter: 1 TABULACIONES
Consulta ejecutada correctamente. localhost (17.0 RTM) sa (80) EcommerceDB 00:00:00
```

Ejercicio 6: Generación de identificadores con SEQUENCE

Se implementó el objeto **SEQUENCE** para generar identificadores numéricos correlativos (**OrderLineID**) de forma independiente a la tabla **OrderDetail**, facilitando la asignación de IDs en transacciones complejas antes de insertar los registros.

Código SQL ejecutado:

SQLQuery2.s...DB (sa (80))*

SQLQuery1.sql...ster (sa (86))*

Novedades

1

USE EcommerceDB;

2

GO

3

4

SELECT * FROM OrderDetail;

91 %

1

0

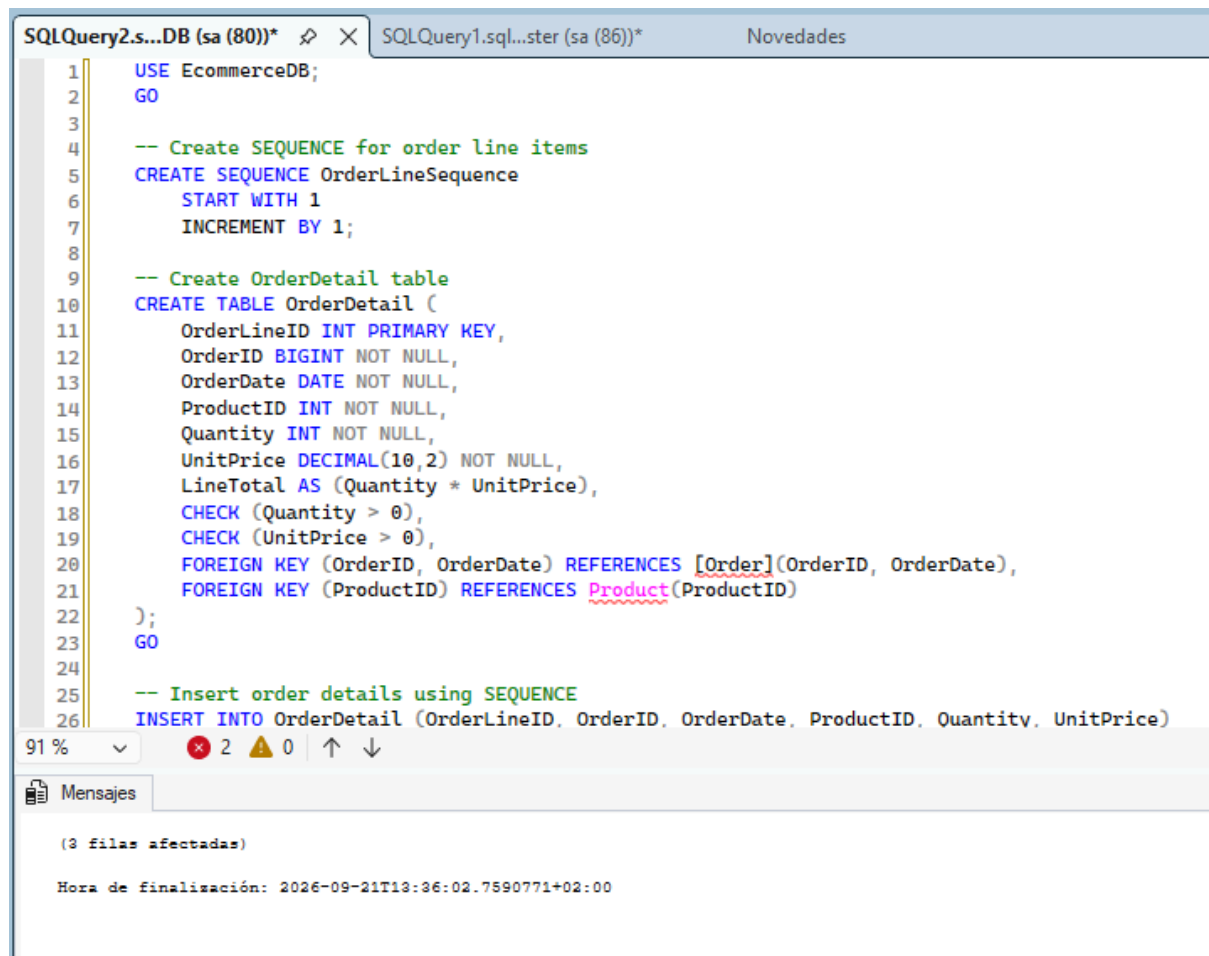
↑

↓

Resultados

Mensajes

	OrderLineID	OrderID	OrderDate	ProductID	Quantity	UnitPrice	LineTotal
1	1	1	2025-01-15	1	2	99.99	199.98
2	2	1	2025-01-15	2	1	149.99	149.99
3	3	2	2025-02-20	1	3	99.99	299.97



The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are two tabs: 'SQLQuery2.s...DB (sa (80))*' and 'SQLQuery1.sql...ster (sa (86))*'. The active tab is 'SQLQuery2.s...DB (sa (80))*'. Below the tabs, there is a 'Novidades' button. The main area displays a SQL script with line numbers 1 through 26. The script includes a 'USE' statement, a 'GO' command, a comment '-- Create SEQUENCE for order line items', a 'CREATE SEQUENCE' statement for 'OrderLineSequence', another 'GO' command, a comment '-- Create OrderDetail table', a 'CREATE TABLE' statement for 'OrderDetail' with various columns and constraints, another 'GO' command, a comment '-- Insert order details using SEQUENCE', and an 'INSERT INTO' statement. Below the script, there is a status bar showing '91 %' and a message icon. Below the status bar, there is a 'Mensajes' tab. The 'Mensajes' tab shows a message: '(3 filas afectadas)' and 'Hora de finalización: 2026-09-21T13:36:02.7590771+02:00'.

```
1  USE EcommerceDB;
2  GO
3
4  -- Create SEQUENCE for order line items
5  CREATE SEQUENCE OrderLineSequence
6      START WITH 1
7      INCREMENT BY 1;
8
9  -- Create OrderDetail table
10 CREATE TABLE OrderDetail (
11     OrderLineID INT PRIMARY KEY,
12     OrderID BIGINT NOT NULL,
13     OrderDate DATE NOT NULL,
14     ProductID INT NOT NULL,
15     Quantity INT NOT NULL,
16     UnitPrice DECIMAL(10,2) NOT NULL,
17     LineTotal AS (Quantity * UnitPrice),
18     CHECK (Quantity > 0),
19     CHECK (UnitPrice > 0),
20     FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
21     FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
22 );
23 GO
24
25 -- Insert order details using SEQUENCE
26 INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice)
```

91 % 2 0

Mensajes

(3 filas afectadas)

Hora de finalización: 2026-09-21T13:36:02.7590771+02:00

Ejercicio 7: Verificación final de objetos e integridad

Se ejecutaron pruebas para confirmar que el motor de base de datos bloquea datos inválidos mediante las restricciones definidas y que el resto de componentes avanzados responden correctamente.

Código SQL ejecutado:

SQLQuery2.s...DB (sa (80))*

SQLQuery1.sql...ster (sa (86))*

Novedades

1

USE EcommerceDB;

2

GO

3

4

-- Verify JSON queries work

5

SELECT ProductName, JSON_VALUE(Metadata, '\$.color') AS Color

6

FROM Product

7

WHERE Metadata IS NOT NULL;

8

9

-- Verify partitioning

10

SELECT \$PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount

11

FROM [Order]

12

GROUP BY \$PARTITION.PF_OrderDate(OrderDate);

13

14

-- Verify temporal table

15

SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime

16

FROM ProductPrice FOR SYSTEM_TIME ALL

17

ORDER BY ProductID, SysStartTime;

91 %

12

0

↑

↓

Resultados

Mensajes

	ProductName	Color
1	Wireless Mouse	blue
2	Cotton T-Shirt	red

	Partition	RecordCount
1	2	2
2	3	1

	ProductID	CurrentPrice	SysStartTime	SysEndTime
1	1	109.99	2026-09-21 11:28:56.2062642	9999-12-31 23:59:59.9999999
2	2	149.99	2026-09-21 11:28:56.2062642	9999-12-31 23:59:59.9999999

Consulta ejecutada correctamente.

localho

SQLQuery2.s...DB (sa (80))*

SQLQuery1.sql...ster (sa (86))*

Novedades

1USE EcommerceDB;

2GO

3

4-- Verify constraints work

5-- This should fail: negative price

6INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)

7VALUES ('Invalid', 1, 1, -50, 10);

91% ▾

● 6 ▲ 0 ⬆ ⬇

Línea: 7, Carácter: 36

TABULACIO

Mensajes

Mens. 547, Nivel 16, Estado 0, Línea 6

Instrucción INSERT en conflicto con la restricción CHECK 'CK_Product_BasePri_534D60F1'. El conflicto ha aparecido en la base de datos 'EcommerceDB', tabla 'dbo.Product', column 'BasePrice'. Se terminó la instrucción.

Hora de finalización: 2026-09-21T19:38:15.5306462+02:00